

Likes and Comments Feature

Overview

This feature adds like and comment functionality to community posts (public exercise submissions) in The Writer's Corner application.

Features Implemented

1. Like Functionality

- **Toggle-able Likes:** Users can like and unlike posts
- **Real-time Updates:** Optimistic UI updates for instant feedback
- **Like Counter:** Displays the number of likes for each post
- **Visual Feedback:** Filled heart icon for liked posts

2. Comment Functionality

- **Add Comments:** Users can post comments on community posts
- **Edit Comments:** Users can edit their own comments
- **Delete Comments:** Users can delete their own comments
- **Comment Counter:** Displays the number of comments for each post
- **Collapsible Section:** Comments can be shown/hidden
- **User Attribution:** Comments show the author's name and timestamp
- **Edit Indicator:** Shows "(edited)" label for modified comments

3. Database Changes

Two new models added to the Prisma schema:

Like Model

- `id` : Unique identifier
- `userId` : Reference to the user who liked
- `submissionId` : Reference to the submission being liked
- `createdAt` : Timestamp
- Unique constraint on (`userId`, `submissionId`) to prevent duplicate likes

Comment Model

- `id` : Unique identifier
- `content` : The comment text
- `userId` : Reference to the comment author
- `submissionId` : Reference to the submission being commented on
- `createdAt` : Timestamp
- `updatedAt` : Timestamp
- Indexes on `submissionId` and `userId` for performance

API Endpoints

Likes

- `POST /api/likes` - Toggle like on a submission
- Body: `{ submissionId: string }`
- Returns: `{ success: boolean, action: 'liked' | 'unliked' }`
- `GET /api/likes?submissionId=<id>` - Get like count and user's like status
- Returns: `{ count: number, liked: boolean }`

Comments

- `GET /api/comments?submissionId=<id>` - Get all comments for a submission
- Returns: `{ comments: Comment[] }`
- `POST /api/comments` - Add a new comment
- Body: `{ submissionId: string, content: string }`
- Returns: `{ success: boolean, comment: Comment }`
- `PUT /api/comments/[id]` - Update a comment
- Body: `{ content: string }`
- Returns: `{ success: boolean, comment: Comment }`
- `DELETE /api/comments/[id]` - Delete a comment
- Returns: `{ success: boolean }`

Components

LikeButton

Location: `components/community/like-button.tsx`

- Displays like button with count
- Handles like/unlike toggle
- Optimistic UI updates
- Toast notifications for feedback

CommentSection

Location: `components/community/comment-section.tsx`

- Collapsible comment section
- Comment form for new comments
- List of existing comments
- Edit/delete buttons for user's own comments
- Inline editing interface

Installation & Setup

1. Database Migration

```
bash
```

```
cd app
npx prisma migrate deploy
# or for development:
npx prisma migrate dev
```

2. Generate Prisma Client

```
bash
npx prisma generate
```

3. Install Dependencies (if needed)

All required dependencies are already in package.json

Usage

The feature is automatically integrated into the community page. Users can:

1. **Like a Post:** Click the heart button below any community post
2. **Unlike a Post:** Click the filled heart button again
3. **Add a Comment:** Click the “Comment” button to expand the comment section, type your comment, and click “Post Comment”
4. **Edit a Comment:** Click the edit icon on your own comment, modify the text, and click “Save”
5. **Delete a Comment:** Click the trash icon on your own comment and confirm deletion

Security

- All endpoints require authentication via NextAuth
- Users can only edit/delete their own comments
- All operations validate that submissions exist and are public
- Database constraints prevent duplicate likes

Design

The UI follows the existing vintage/typewriter theme with:

- Ink (dark text)
- Rust (accent color for interactions)
- Forest (secondary text)
- Sepia/Gold (backgrounds and highlights)
- Typewriter font for headers
- Serif font for body text

Future Enhancements

Possible improvements for future iterations:

- Nested replies to comments
- Mention system (@username)
- Rich text formatting in comments
- Like/comment notifications
- Comment moderation tools
- Reaction types beyond just “like”